

Programación II

Introducción a la Programación Orientada a Objetos (POO)

POO

Es un paradigma de programación que utiliza "objetos" para diseñar aplicaciones y programas. Un objeto es una instancia de una clase, que es una plantilla o blueprint que define las propiedades y comportamientos (métodos) comunes.

Programación Estructurada VS POO

Programación Estructurada:

- Se basa en una secuencia de procedimientos o funciones.
- El enfoque está en las funciones y la lógica.
- Los datos y las funciones están separados.
- Ejemplo: C

Programación Orientada a Objetos:

- Se basa en objetos que contienen datos y métodos.
- El enfoque está en los objetos y la interacción entre ellos.
- Los datos y los métodos que operan sobre esos datos están encapsulados juntos.
- Ejemplo: Java

Beneficios de la Aplicación de la POO



Reusabilidad: Se pueden reutilizar clases y objetos en diferentes programas.

Mantenibilidad: El código es más fácil de mantener y actualizar.

Modularidad: El código está organizado en módulos (clases) independientes.

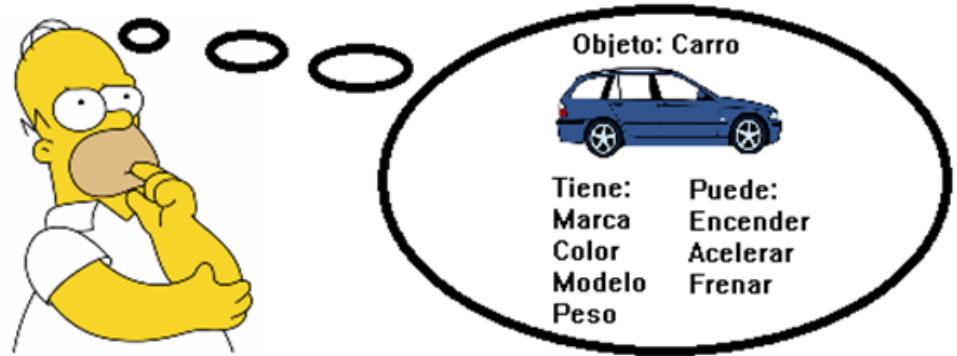
Escalabilidad: Facilita la ampliación y modificación de aplicaciones grandes.

Programación I

Principios de la Programación Orientada a Objetos

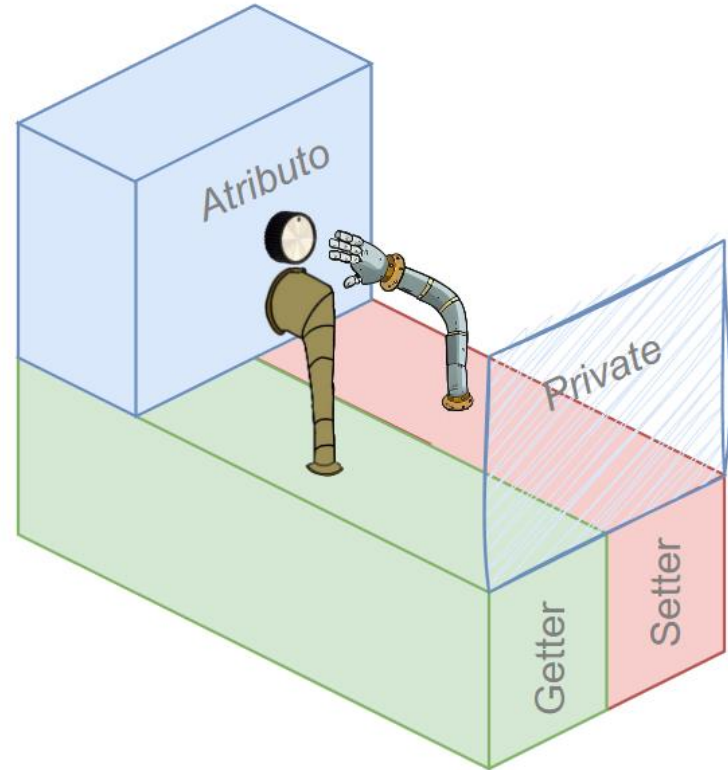
Abstracción

Ocultar los detalles complejos mostrando solo la información esencial.



Encapsulamiento

El encapsulamiento en Java es el concepto de ocultar los detalles internos de una clase y proporcionar una interfaz pública para interactuar con la clase. Esto se logra mediante el uso de modificadores de acceso como `private`, `public` y `protected`, así como mediante el uso de métodos de acceso (getters) y métodos de modificación (setters) para acceder a los datos de la clase.



Encapsulamiento

```
ej06ClaseEjemplo.java
src > CursoDeJava

public class Persona {
    private String nombre;
    private int edad;

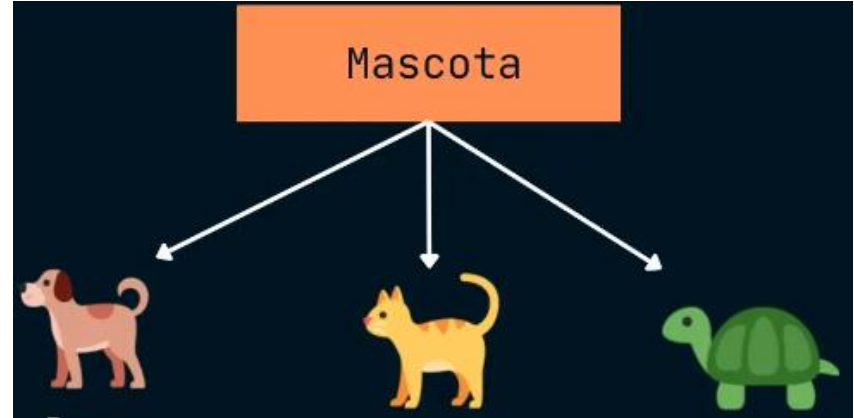
    public Persona(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }

    public String getNombre() {
        return nombre;
    }

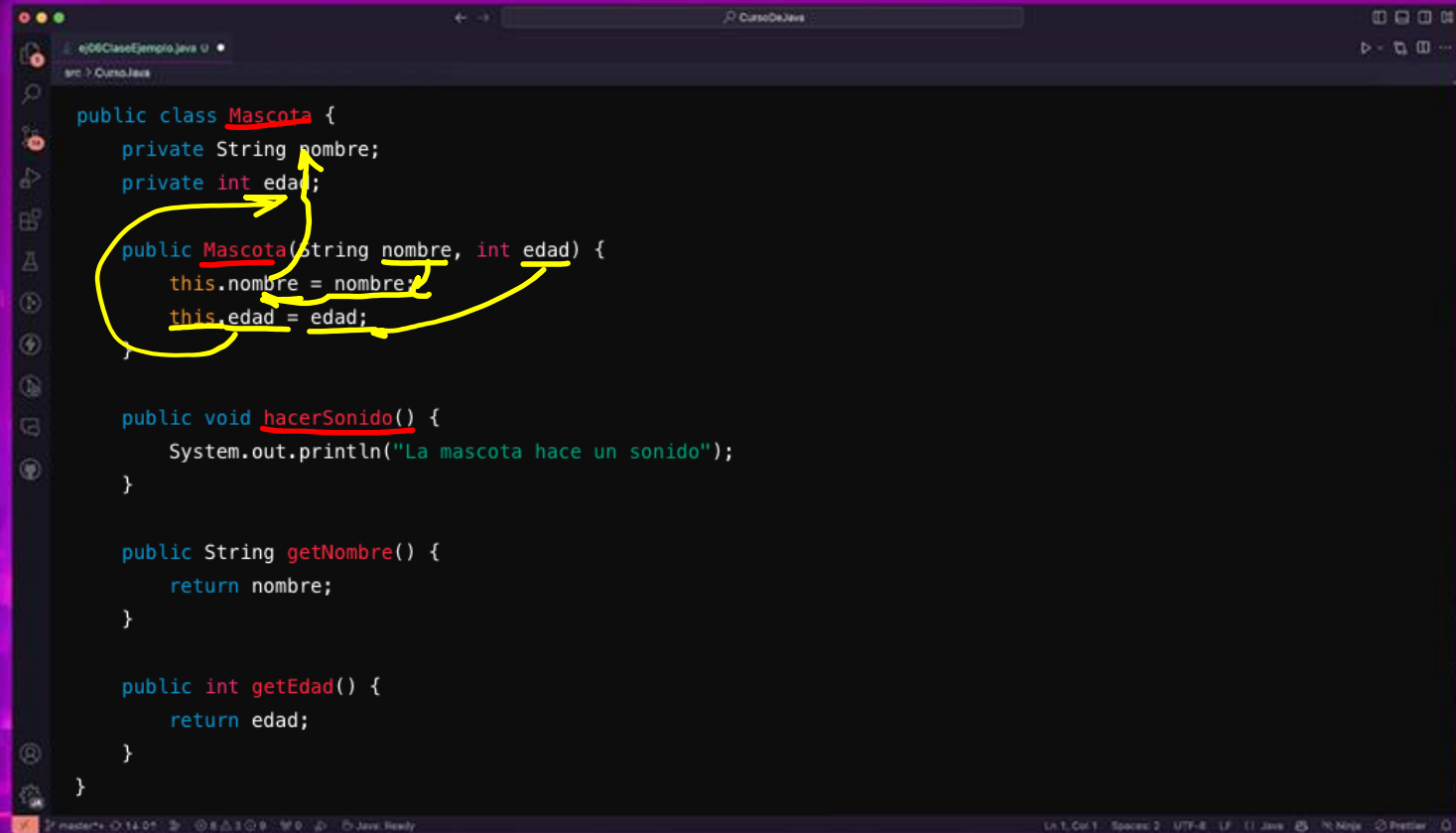
    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
}
```

Herencia

Permite crear nuevas clases basadas en clases existentes. La clase nueva, llamada subclase o clase derivada, hereda los atributos y métodos de la clase existente, llamada superclase o clase base. Esto facilita la reutilización del código y establece una relación "es-un" entre las clases.



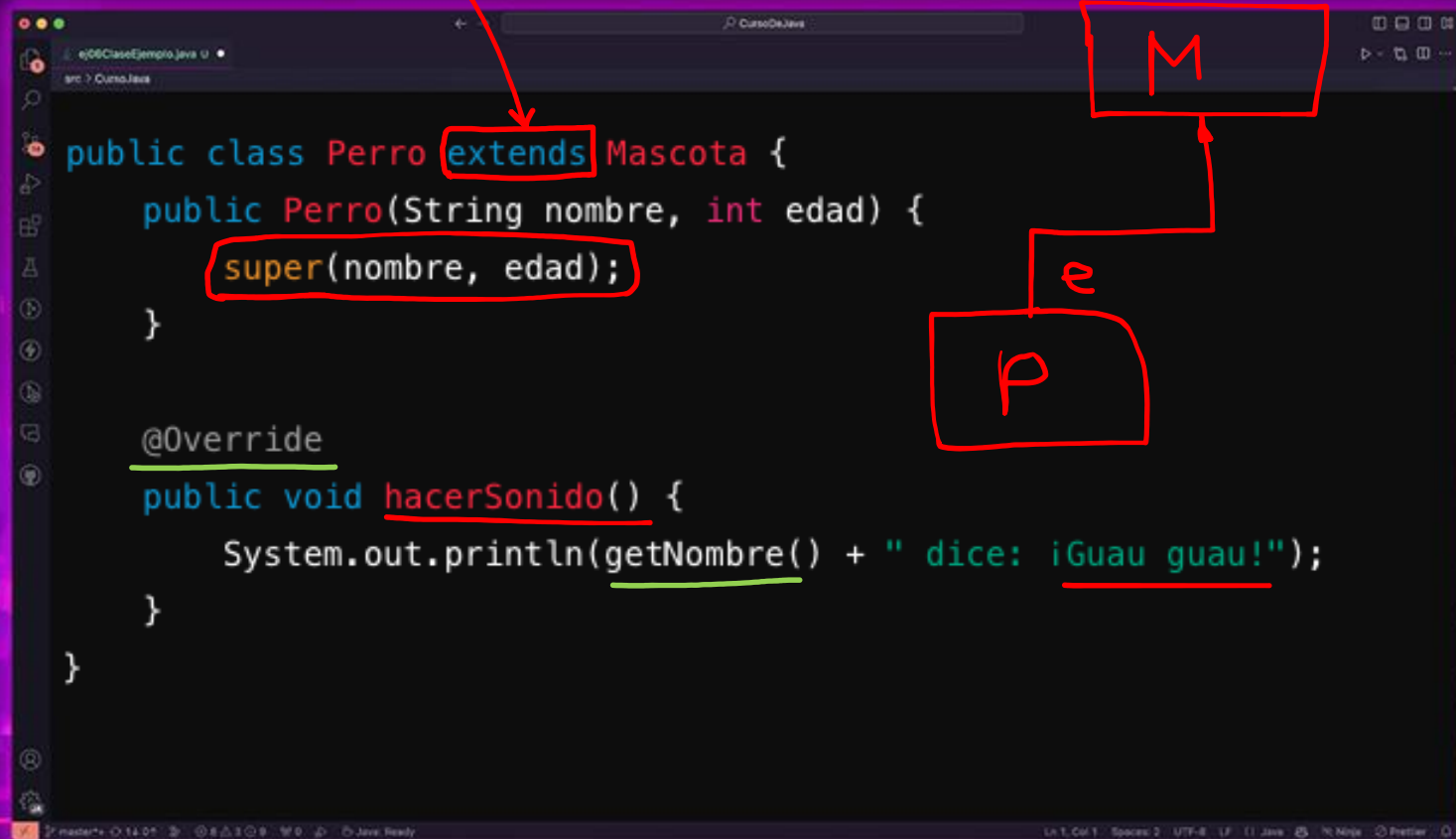
Herencia



```
public class Mascota {  
    private String nombre;  
    private int edad;  
  
    public Mascota(String nombre, int edad) {  
        this.nombre = nombre;  
        this.edad = edad;  
    }  
  
    public void hacerSonido() {  
        System.out.println("La mascota hace un sonido");  
    }  
  
    public String getNombre() {  
        return nombre;  
    }  
  
    public int getEdad() {  
        return edad;  
    }  
}
```

The image shows a code editor window with a Java class named `Mascota`. The class has two private attributes: `String nombre` and `int edad`. It has a constructor `Mascota(String nombre, int edad)` that initializes these attributes using `this.nombre = nombre;` and `this.edad = edad;`. There are also two methods: `hacerSonido()` which prints a message, and two getter methods `getNombre()` and `getEdad()`. Yellow annotations are present: a circle around the constructor, arrows pointing from `this.nombre` and `this.edad` to the class name `Mascota`, and arrows pointing from `nombre` and `edad` in the getters to the class name `Mascota`. The editor interface includes a sidebar with icons, a top bar with the file name `ej06ClaseEjemplo.java` and the editor `CursoDeJava`, and a bottom status bar showing `Ln 1, Col 1`, `Spaces: 2`, `UTF-8`, `LF`, `Java`, and `Prettier`.

Herencia



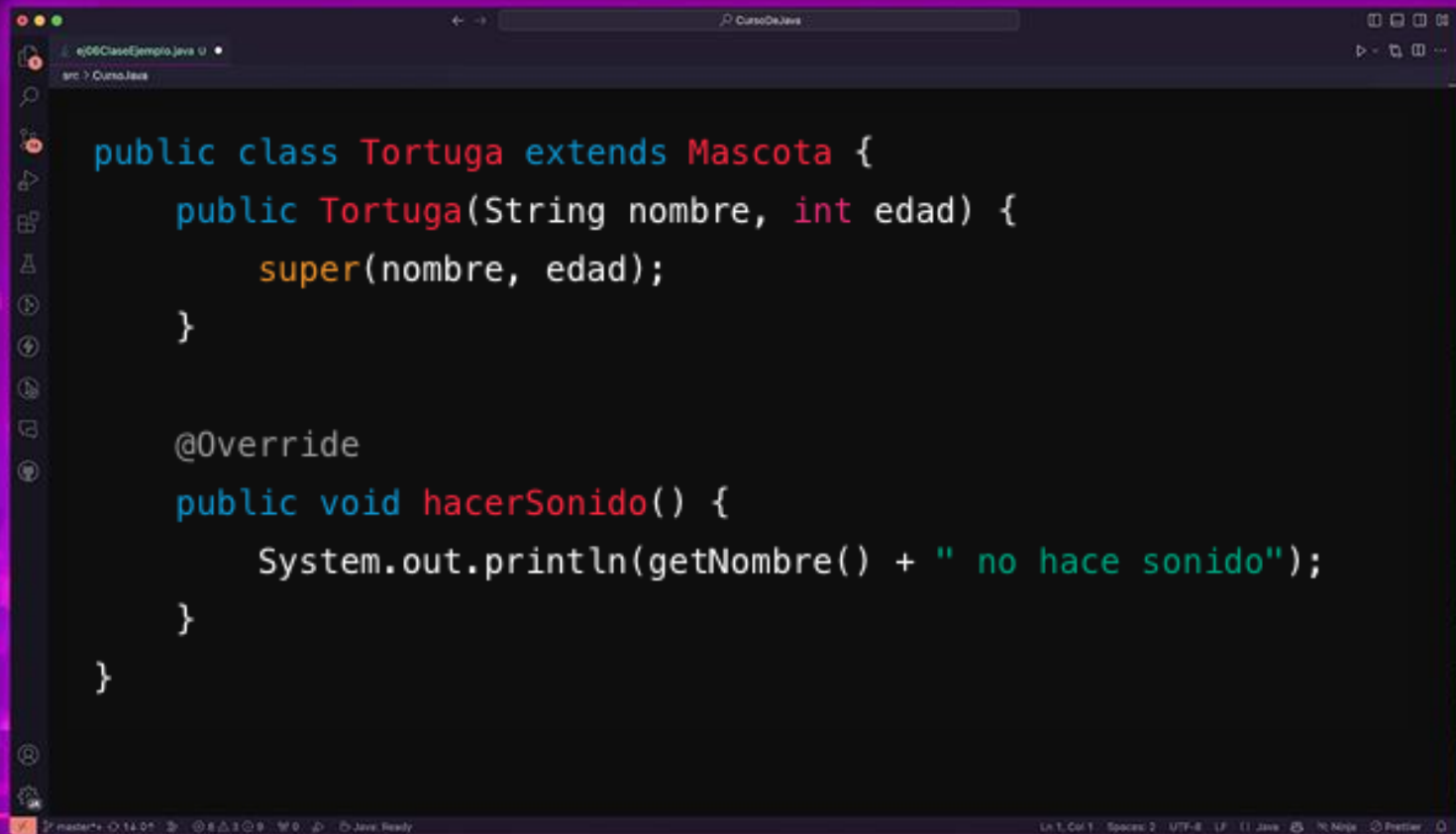
Herencia

```
ej06ClaseEjemplo.java
src > CursoDeJava

public class Gato extends Mascota {
    public Gato(String nombre, int edad) {
        super(nombre, edad);
    }

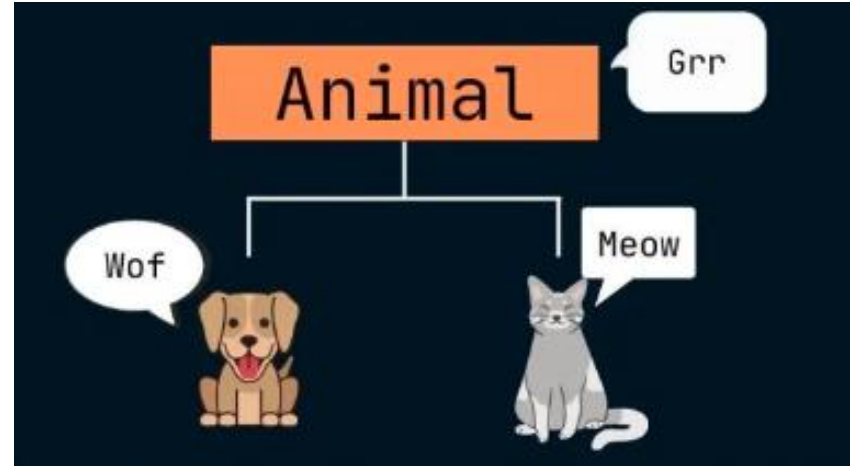
    @Override
    public void hacerSonido() {
        System.out.println(getNombre() + " dice: ¡Miau miau!");
    }
}
```

Herencia



Polimorfismo

Capacidad de objetos de diferentes clases de responder al mismo mensaje o llamada a un método de manera distinta, según la implementación específica de cada clase. En otras palabras, el polimorfismo permite que un objeto pueda comportarse de múltiples formas.



Polimorfismo

```
ej06ClaseEjemplo.java
src > CursoDeJava

class Mascota {
    public void sonido() {
        System.out.println("Sonido de una mascota");
    }
}

class Tortuga extends Mascota {
    @Override
    public void sonido() {
        System.out.println("Sonido de una tortuga: ¡Silencio!");
    }
}

class Perro extends Mascota {
    @Override
    public void sonido() {
        System.out.println("Sonido de un perro: ¡Guau!");
    }
}
```

Clases

Es un molde para crear objetos. Define un tipo de objeto según sus atributos (datos) y métodos (funciones).

Es un modelo que especifica qué atributos y métodos tendrán los objetos creados a partir de esa clase.

Clases

```
ej06ClaseEjemplo.java
src > CursoDeJava

public class Persona {
    // Atributos (propiedades o variables de instancia)
    private String nombre;
    private int edad;

    // Constructor
    public Persona(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }

    // Métodos (funciones o comportamientos)
    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
}
```

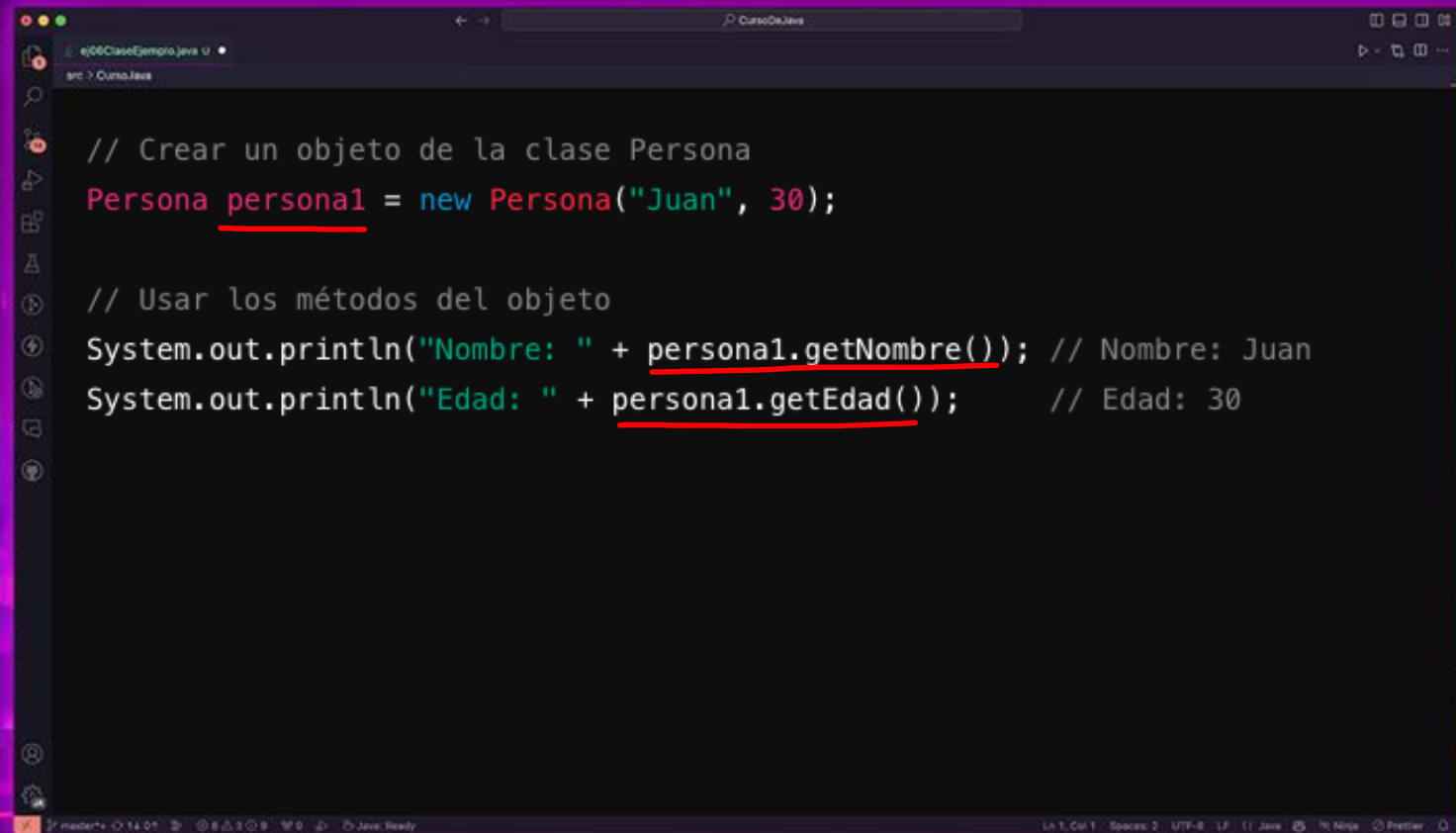
Objeto

Es una instancia de una clase.

Es una entidad concreta que tiene un estado y un comportamiento definido por la clase a partir de la cual se creó.

Cada objeto tiene su propio conjunto de valores para los atributos definidos en su clase.

Objeto



```
// Crear un objeto de la clase Persona
Persona persona1 = new Persona("Juan", 30);

// Usar los métodos del objeto
System.out.println("Nombre: " + persona1.getNombre()); // Nombre: Juan
System.out.println("Edad: " + persona1.getEdad());      // Edad: 30
```

The image shows a code editor window with a dark theme. The title bar indicates the file is 'ej06ClaseEjemplo.java'. The code is in Java and demonstrates object creation and method usage. The variable 'persona1' and the method calls 'getNombre()' and 'getEdad()' are underlined in the original image. The editor interface includes a sidebar with icons for Explorer, Search, Source Control, and Run and Debug. The status bar at the bottom shows 'Ln 1, Col 1', 'Spaces: 2', 'UTF-8', 'LF', 'Java', and 'Prettier'.

Diferencias entre Clases y Objetos

Clase: Define un tipo de objeto con atributos y métodos.

Objeto: Una instancia de una clase que tiene un estado particular y puede ejecutar métodos definidos en la clase.

Diferencias entre Clases y Objetos

Definición vs. Instancia

Clase

Es una definición, una plantilla. No es una entidad tangible.

Objeto

Es una instancia concreta de una clase. Es una entidad tangible que ocupa memoria y tiene un estado específico.

Diferencias entre Clases y Objetos

Abstracción vs. Concreción

Clase

Representa la **abstracción** de algo. Define qué características y comportamientos deberían tener los objetos.

Objeto

Representa la **concreción** de esa abstracción. Tiene características y comportamientos específicos definidos por la clase.

Diferencias entre Clases y Objetos

Reusabilidad

Clase

Puede ser utilizada para crear múltiples objetos.

Objeto

Es una única instancia de una clase específica.

Diferencias entre Clases y Objetos

Estructura vs. Estado

Clase

Define la estructura (atributos) y el comportamiento (métodos) comunes a todos los objetos de ese tipo.

Objeto

Tiene un estado particular (valores de atributos) y puede ejecutar comportamientos definidos por su clase.



this

La palabra clave `this` en Java se refiere a la instancia actual de una clase. Tiene varios usos importantes en la programación orientada a objetos

Referencia al objeto actual

```
public class Persona {  
    private String nombre;  
  
    public Persona(String nombre) {  
        this.nombre = nombre; // Aquí, 'this.nombre' se refiere al atributo  
    }  
}
```

this se utiliza para hacer referencia al objeto actual dentro de la propia clase. Puede ser útil cuando hay ambigüedad entre los nombres de variables de instancia y parámetros en un método.

Llamada al constructor de la misma clase

```
public class Persona {  
    private String nombre;  
    private int edad;  
  
    public Persona(String nombre) {  
        this(nombre, 0); // Llama al otro constructor de Persona con dos parámetros  
    }  
  
    public Persona(String nombre, int edad) {  
        this.nombre = nombre;  
        this.edad = edad;  
    }  
}
```

this se puede usar para llamar a otro constructor de la misma clase desde un constructor. Esta técnica se llama "reutilización de constructores" o "llamada al constructor".



Llamada al constructor de la misma clase

```
public class MiClase {  
    private int dato;  
  
    public void procesarDato() {  
        // Aquí llamamos a otro método y pasamos 'this' como argumento  
        otroMetodo(this);  
    }  
  
    private void otroMetodo(MiClase instancia) {  
        // Hacer algo con la instancia actual  
    }  
}
```

En algunos casos, es útil pasar la instancia actual como argumento a otro método.

Sobrecarga de métodos

Métodos con el Mismo
Indicador y Distinto Número
de Parámetros

Sobrecarga de métodos

```
class Matematica {  
    public int sumar(int a, int b) {  
        return a + b;  
    }  
  
    public int sumar(int a, int b, int c) {  
        return a + b + c;  
    }  
}
```

En Java, se pueden definir múltiples métodos con el mismo nombre pero diferentes listas de parámetros.

Métodos Estáticos

Los métodos estáticos pertenecen a la clase, no a una instancia de la clase. Se pueden llamar sin crear un objeto de la clase.

Métodos Estáticos

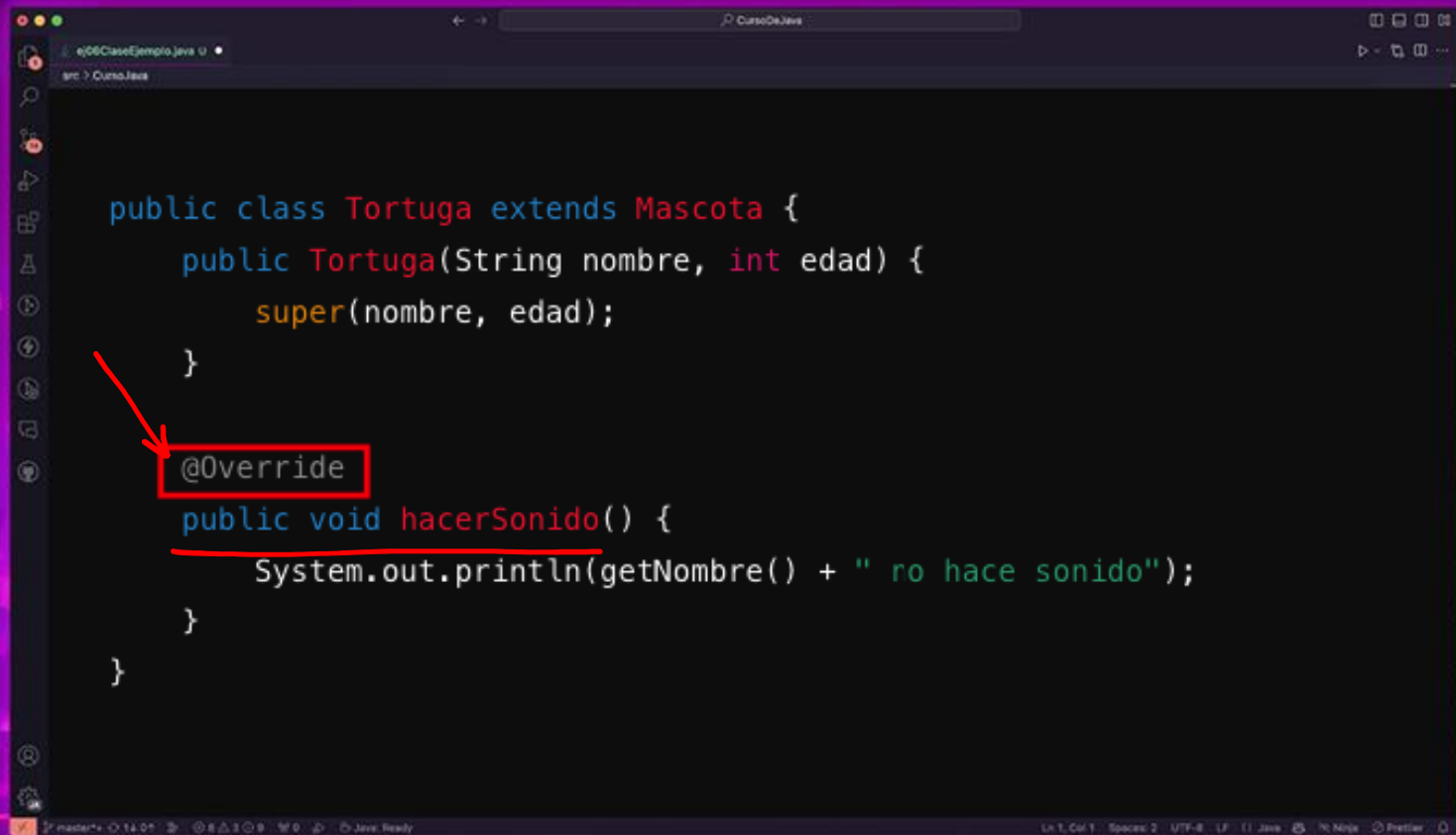
```
class Utilidad {  
    public static int multiplicar(int a, int b) {  
        return a * b;  
    }  
}  
  
public class TestUtilidad {  
    public static void main(String[] args) {  
        System.out.println(Utilidad.multiplicar(3, 4)); // 12  
    }  
}
```

Se pueden llamar sin crear un objeto de la clase.

Sobrescritura

La sobrescritura (@Override) permite que una subclase proporcione una implementación específica de un método que ya está definido en su superclase.

Sobrescritura



Recursividad de Métodos

Un método recursivo es un método que se llama a sí mismo para resolver un problema.

Recursividad de Métodos

```
ej06ClaseEjemplo.java
src > CursoDeJava

class Matematica {
    public static int factorial(int n) {
        if (n == 0) {
            return 1;
        }
        return n * factorial(n - 1);
    }
}

public class TestMatematica {
    public static void main(String[] args) {
        System.out.println(Matematica.factorial(5)); // 120
    }
}
```

Factorial de un
Número

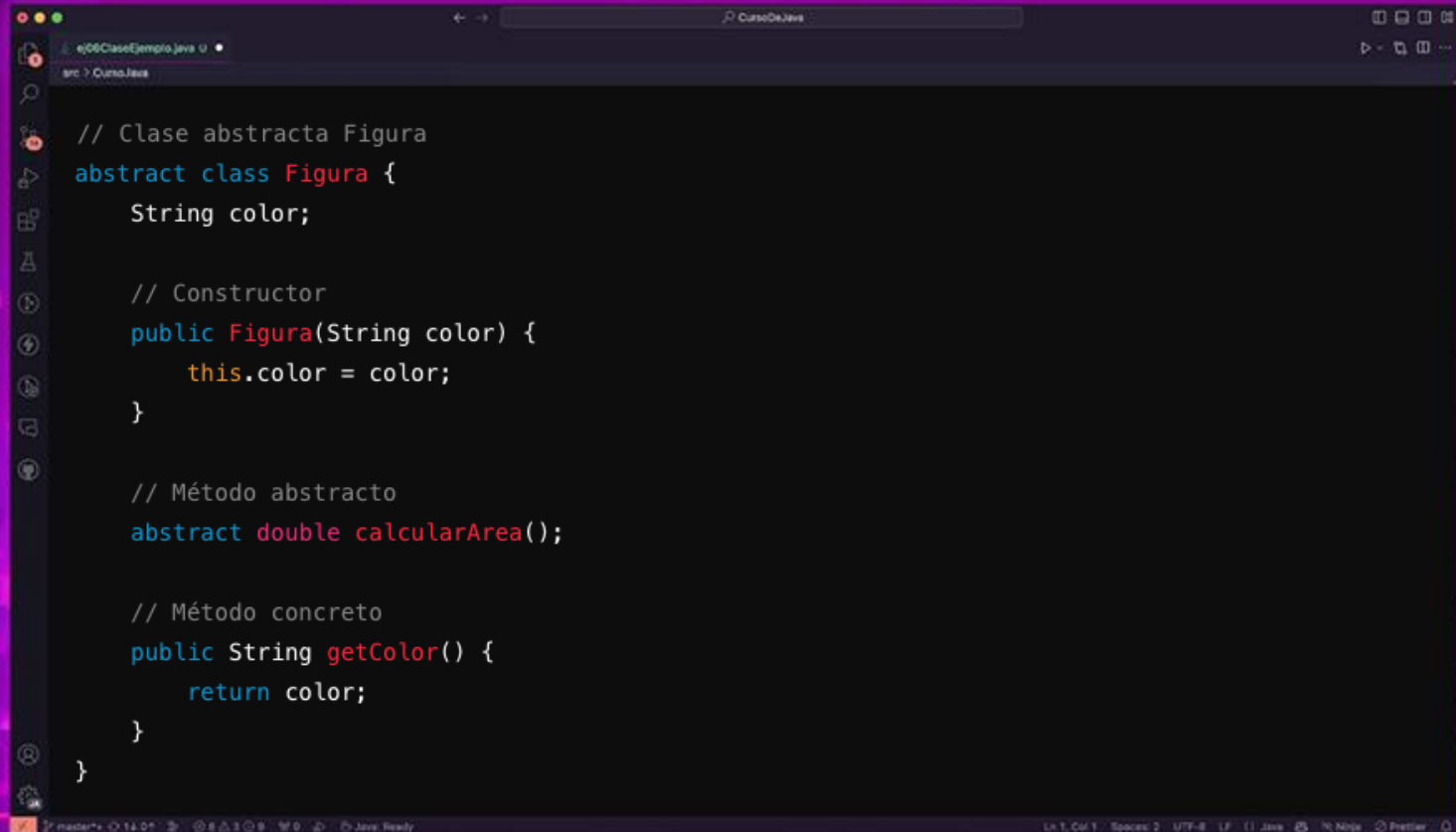
Clases Abstractas

Una clase abstracta en Java es una clase que no se puede instanciar, es decir, no se puede crear un objeto directamente a partir de ella. Las clases abstractas están diseñadas para ser extendidas por otras clases que implementan los métodos abstractos declarados en la clase abstracta. Una clase abstracta puede contener métodos abstractos (sin implementación) y métodos concretos (con implementación).

Características de las Clases Abstractas

1. **No se pueden instanciar:** No puedes crear objetos de una clase abstracta.
2. **Métodos abstractos:** Puede tener métodos abstractos, que son métodos sin cuerpo que deben ser implementados por las clases derivadas.
3. **Métodos concretos:** Puede tener métodos concretos, que son métodos con cuerpo.
4. **Constructores:** Aunque no se puede instanciar, una clase abstracta puede tener constructores que pueden ser llamados por sus subclases.

Clase Abstracta Básica



The image shows a code editor window with a dark theme. The title bar at the top says "CursoDeJava". The editor contains the following Java code for an abstract class named `Figura`:

```
// Clase abstracta Figura
abstract class Figura {
    String color;

    // Constructor
    public Figura(String color) {
        this.color = color;
    }

    // Método abstracto
    abstract double calcularArea();

    // Método concreto
    public String getColor() {
        return color;
    }
}
```

The code is color-coded: keywords like `abstract`, `class`, `public`, and `return` are in blue; the class name `Figura` is in red; and the method name `calcularArea` is in red. The variable `color` is in green. The editor has a sidebar on the left with icons for Explorer, Search, Source Control, and Run and Debug. The status bar at the bottom shows "Ln 1, Col 1", "Spaces: 2", "UTF-8", "LF", "Java", and "Prettier".

Clase Abstracta Básica

```
ej06ClaseEjemplo.java
src > CursoDeJava

// Subclase Circulo
class Circulo extends Figura {
    double radio;

    public Circulo(String color, double radio) {
        super(color);
        this.radio = radio;
    }

    @Override
    double calcularArea() {
        return Math.PI * radio * radio;
    }
}
```

Clase Abstracta Básica

```
ej06ClaseEjemplo.java
src > CursoDeJava

// Subclase Rectangulo
class Rectangulo extends Figura {
    double ancho, alto;

    public Rectangulo(String color, double ancho, double alto) {
        super(color);
        this.ancho = ancho;
        this.alto = alto;
    }

    @Override
    double calcularArea() {
        return ancho * alto;
    }
}
```

Ln 1, Col 1 | Spaced 2 | UTF-8 | LF | 1 Java | 0 Nixie | Prettier

Clase Abstracta Básica

```
ej06ClaseEjemplo.java
src > CursoDeJava

// Clase principal para probar las clases anteriores
public class Main {
    public static void main(String[] args) {
        Figura circulo = new Circulo("Rojo", 5);
        Figura rectangulo = new Rectangulo("Azul", 4, 6);

        System.out.println("El área del círculo es: " + circulo.calcularArea());
        System.out.println("El área del rectángulo es: " + rectangulo.calcularArea());
    }
}
```